Acing the Amazon SDE Interview

Sample Interview Topics

Below is a list of broad areas that we expect people to be familiar with. It is certainly not required that you memorize all of the information outlined below, but this should serve as a helpful reference guide for the types of things you might want to brush up on before interviewing with Amazon.

Programming Languages

We do not require that you know any specific language before interviewing for a technical position at Amazon.com, but familiarity with a prominent object-oriented language is generally a prerequisite for success. Not only should you be familiar with the syntax of a language like C++, Java, or C#, but you should also know some of the language's nuances such as how memory management works, what some of the most commonly used collections or libraries are, etc. You should be able to compare languages and talk about the tradeoffs between using language X vs. language Y. Additionally, it is considered a plus to be familiar with some scripting language such as perl, ruby, awk, etc. It is also nice to know the basics of regular expression as it is now a mainstay in both the object oriented and scripting worlds.

Data Structures

Most of the work we do involves storing and providing access to data in efficient ways. This necessitates a very strong background in standard data structures. You should know what each of these data structures is and how they are implemented, what their runtimes are for common operations, and under what circumstances it would be beneficial to use a particular one. The list below is in no particular order:

- Array
- Linked List
- Tree (Tree, Binary Tree, Binary Search Tree, Red-Black Tree, etc.)
- Heap
- Hash Table
- Stack
- Queue
- Tree
- Graph (both directed and undirected)
- Algorithms

It is also important to know efficient ways to manipulate data. One great way of demonstrating this is to brush up on some common algorithms. We will expect that you can apply and discuss the tradeoffs between commonly used algorithms.

- Sorting: Bubble Sort; Merge Sort; Quick Sort; Radix/Bucket Sort
- Traversals (On multiple data structures): Depth First Search; Breadth First Search

Coding
Expect to be asked to code syntactically correct code– no pseudo code. If you are a bit rusty coding without an IDE or coding in a specific language, it is probably a good idea to dust off the cobwebs and get comfortable coding with pen and paper. The most important thing a software engineer does at Amazon.com is write scalable, stable, robust, and well tested code. These are going to be the main criteria by which your code will be evaluated, so make sure that you check for edge cases and common error inputs, as well as the "happy paths" through the code.

Object Oriented Design
Good design is paramount to extensible, bug free, and long living code. It is possible to solve software problems in almost limitless numbers of ways; but when software needs to be robust and extensible,
It is important to know some common techniques that help with this. Using object oriented design best practices is one way to build lasting software. You should have a working knowledge of a few common and useful design patterns (singleton, factory, adapter, bridge, visitor, command, proxy, observer, etc.), as well as know how to write software in an object oriented way with appropriate use of inheritance and aggregation.

Databases
Most of the software that we write is backed by a database somewhere. A lot of the challenges we face come into play when interfacing with existing data models and when designing new data models. You should know the basics of how relational databases work, how to design relational database schemas, and how to write basic SQL queries against a database.

Distributed Computing
Our systems at Amazon.com usually have to work under very strict tolerances at high load. While we have some internal tools that help us with scaling, it is important to have an understanding of a few basic distributed computing concepts. Having an understanding of topics such as map-reduce, service oriented architectures, distributed caching, and load balancing, will help you in formulating answers to some of the more complicated distributed architecture questions you might encounter.

Internet Topics
This is Amazon.com; we are an online company and we expect our engineers to be at least familiar with the basics of how the internet works. You might want to brush up on how internet browsers do what they do, DNS lookups, what TCP/IP and HTTP are, sockets, etc. We are not necessarily looking for network engineering types of qualifications, but a solid understanding of the fundamentals of how the web works is a requirement.

Operating Systems
You will not need to know how to build your own operating system, but you should be familiar with some OS topics that can affect code performance; things such as memory management, processes, threads, synchronization, paging, multithreading, deadlocks (causes, detection, avoidance).

<u>There will be 3 Coding Sessions:</u>

1. ***Problem Solving***
    a. Did you correctly solve the problem?
    b. Did you reach an optimal solution?
    c. Can you justify decisions by identifying potential tradeoffs with several different solutions?
    d. Do you understand why one solution is better than another?
    e. Did you define the problem by asking relevant clarifying questions?

f. Did you consider additional factors beyond the basic problem like developer effort, or team composition?
g. Do you require minimal hints and able to use the hints effectively?
h. How do you consider edge cases or difficult changes to the requirements?
i. How do you react when your assumptions are challenged?

2. ***Logical & Maintainable Code***
   a. Is your code simple (leverages reuse, properly formatted, no improper coding constructs)?
   b. Is your code maintainable? (quickly able to trace impact of changes, clear variable naming conventions)?
   c. Is your code clear and understandable?
   d. Is your code following proper and consistent syntax?
   e. Does your code work as intended?
3. ***Data Structures & Algorithms***
   a. Do you know how to apply proper data structures & algorithms to solve a technical problem
   b. Do you know the tradeoffs of different data structures (ex. hash map vs tree vs array)
   c. Do you know the tradeoffs of different algos (breadth search vs depth search)
   d. Can you justify why the selected data structure and algorithm was used?
   e. Do you know how to translate these into a workable solution?
   f. Do you know the run time and space complexity tradeoffs of the different solutions?
   g. Can you optimize the solution?

<u>General Tips for Doing Well on the Coding Questions:</u>

Before Coding:

1. Ask clarifying questions
2. Voice your assumptions – this will help the interviewer steer you back on track if you're making fundamentally wrong assumptions before you start coding
3. Ask for examples and use them to validate your ideas

During Coding:

1. Explain your decision making
2. Describe your code
3. Write syntactically correct real code NEVER pseudocode. Try to be consistent in your naming conventions and it will make testing and optimizing easier later.
4. If you forget a specific API or specific link into an array or something like that, just ask the interviewer who should help you

After Coding:

1. Test your code
2. Talk through optimizing the solution if it can be further optimized if there is more time

Most important here is that you should focus on having a working and fully coded solution on the board to the problem by the end of each interview. If it's easier for you to start with an inefficient brute force approach and then optimize from there at the end, it's a perfectly valid approach. Just state that up front with your interviewer. Brute force solution is better than an incomplete solution.